Timing System Design for the NSLS-II

Prepared by: Joe DeLong

July 2009

Contents

1. Scope…… 3
2. Introduction…….. 4
3. Requirements…… 5
   3.1. Operations……. 5
   3.2. Infrastructure …… 5
   3.3. Software…… 5
   3.4. Major Subsystems requirements…… 6
      3.4.1. Power Supplies…… 6
      3.4.2. Pulsed Power…… 6
      3.4.3. Beam Position Monitors……... 7
      3.4.4. Vacuum Systems…….. 8
      3.4.5. Diagnostics……... 8
      3.4.6. Radio Frequency…….. 8
      3.4.7. Protection Systems……... 9
4. Theory of operation…….……... 10
   4.1. Structure of the Timing System……. 10
   4.2. Event Generator……. 10
      4.2.1. Event Sources…….. 10
         4.2.1.1. Sequencer…… 10
         4.2.1.2. External triggers…….. 10
         4.2.1.3. Multiplexed Counters…….. 10
         4.2.1.4. AC Line Synchronization…… 10
      4.2.2. Distributed Bus…… 11
      4.2.3. Data Buffer ……..…… 11
      4.2.4. Time Stamping…… 12
   4.3. Event Receiver…… 13
      4.3.1. Trigger Outputs……... 13
         4.3.1.1. TTL…….. 13
         4.3.1.2. CML…… 13
         4.3.1.3. PECL…….. 14
      4.3.2. Prescaler Outputs…… 14
      4.3.3. TTL set / reset latch Outputs…….. 14
      4.3.4. Distributed Bus Outputs……. 14
      4.3.5. Data buffer……. 14
      4.3.6. IOC interrupts……. 15
      4.3.7. Time Stamping…… 16
      4.3.8. Local event external input…….. 17
      4.3.9. Interlock Input…… 18

4.4. General Machine Time…….. 19
5. System components…….. 11
6. Infrastructure and layout…… 6
7. Software……. 7
7.1. Event Generator…….. 8
7.2. Event Receiver…… 9
7.2.1. VME…… 10
7.2.2. cPCI…… 11
7.2.3. PMC…… 12
7.3. Universal IO modules…… 13
7.3.1. TTL input module…… 14
7.3.2. TTL output module……. 15
7.3.3. PECL output module…….. 16
7.3.4. PECL output module with fine delay control……. 17
7.4. Time Standard (Global Machine Time)……. 14
7.5. RF clock…….. 15
8. Software development……..… 16
8.1. RTEMS
8.2. EPICS
9. System monitoring ……..…. 16

1. Scope

This Document will describe in detail the hardware and software used in the NSLS-II timing system. As well as cover the details of making changes to the software and plans for the future.

Section 2: Introduces the general requirements of the timing system.

Section 3: System requirements from each of the major technology systems.

Section 4: Theory of operation, describes how the system works in a general way.

Section 5: Describes the hardware components in detail.

Section 6: Covers the conventional facilities and the placement of system infrastructure.

Section 7: Details the software drivers for all components used in the timing system.

Section 8: Details the software development process.

Section 9: Describes the methods for monitoring the operational system for failures.

2. Introduction

The Timing system for the NSLS-II will deliver clocks, triggers and time stamp information to all technology subsystems and experimental areas in the light source. Various systems have different timing requirements and hardware will be developed to fulfill these distinct needs.

The basis for the timing system is an event link with machine event information transmitted synchronously with respect to the RF system clock. This link is distributed throughout the complex and, where needed, will be delivered simultaneously and with a constant delay with respect to a central distribution point. The distribution media is multi-mode fiber. The fibers will be run in a multi star topology.

System events will be generated under software control in the EVent Generator. This is a VME card with inputs from the RF system and the 60Hz line frequency. In addition to software events there are eight hardware trigger inputs and an additional bus for timing, machine state and data transfer.

System events will be decoded and local triggers will be generated by the EVent Receivers. These devices will be available in several form factors including VME, Compact PCI and PMC. In addition to receiving system events these devices will generate time stamp information for the subsystems. Other capabilities of this device will be discussed in later sections of this document.

The Global Machine Time base establishes a reference time for all time stamping in the complex. This device will generate a 1 second clock as well as a 10MHz clock that will be distributed to the technology systems. This same hardware will be used as a network time server for synchronization of applications. Precise time of day information will be available from the EVRs such that post mortem analysis will be possible.

Software for the timing system will be developed to create an intuitive interface to timing hardware. A stand alone application will be developed for the EVG and EVRs. An application for entering a machine super cycle will also be developed to control the EVG.

3. Requirements

3.1. Operations

The NSLS-II requires a robust and scalable distributed timing system. The addition of control points must be as easy as adding the required off the shelf hardware and configuring the software. This operation should take no more than a day for installation, configuration and test.

Hardware availability must be maintained such that unforeseen requirements can be filled and hardware failures can be repaired immediately. A cache of spares needs to be in place prior to commissioning. The quantity of spares will be determined from the experiences at other light sources.

Tracking of failures and associated down time will be done so that system improvements can be made to maximize machine run time.

3.2. Infrastructure

To implement this type of scalable architecture the infrastructure must be in place prior to commissioning the machine. The distribution network must have 100% coverage in all areas where timing signals may be needed.

Fiber optic cables to each cell must deliver the signals from the timing system simultaneously. Matched length cables will be installed from a central distribution point near the RF control room. Variation of the arrival time must be kept below 50ps with respect to the bunched beam in the machine.

The fundamental time base will be provided by the RF group. This 500MHz clock will also be distributed as required to experimenters so that fine timing can be generated locally. The output of the 500MHz master oscillator is assumed to be the input for timing. This signal may be manipulated by the LLRF system downstream of the timing system so it may not have a constant phase with respect to the beam. This needs to be resolved.

3.3. Software

Two levels of timing control will be developed to implement an intuitive interface to the timing hardware. The first is a super cycle generator used to create a sequence of events associated with the process of generating, accelerating, storing and topping off the beam. The second level is a software module or

methodology that gives control to programs for manipulating individual trigger channels.

Control of timing for each subsystem must be simple, intuitive and embedded into the high level (user) software of that system. A simple consistent interface will be developed for use in these programs.

Careful archiving will be done to recall machine settings and revert back to a known good machine state. This archiving will be done automatically on a 60 second interval as well as on a user command or when a change is made to a parameter of the system.

More details about software integration will be added after reviewing the approaches taken at other machines.

3.4. Major subsystem requirements

3.4.1. Power Supplies

Standard events from the timing system are required for the power supply controllers as well as triggers that are synchronized to the 60Hz line frequency. These events require a time resolution of < 10ns and jitter of less than 20ps RMS. Each cell controller will require eight TTL timing triggers from one event receiver. These signals need to drive a 50Ω load. Proper distribution and termination of these signals will be determined and, if required, fan-out modules will be provided as system development proceeds.

3.4.2. Pulsed Power Supplies

Extraction power supplies in the Booster and injection power supplies in the Storage ring require precise timing so that the beam can be extracted and injected cleanly. This requires timing pulses with a resolution of less than 500ps And jitter of less than 5ps.

One event receiver for the Booster extraction equipment and one for the Storage ring injection equipment have been requested. This will provide up to 14 triggers for each.

3.4.3. Beam Position Monitors including fast and slow orbit feedback (Libera)

Beam Synchronous timing is required for the BPM system. Triggers for this system will be synchronized to the beam through the RF clock. The timing system must be able to trigger on a specified bunch in the machine. Clocks provided to this system must have phase resolution of less than the bunch frequency. The identified signals and their specifications are listed:

3.4.3.1. System Clock

A 10MHz system clock is required for time stamp information. Global time stamping will be generated from this reference. A GPS disciplined real time clock will be used for this purpose (section 5). Distribution through the timing system will cause +/- 2ns of jitter however this should be sufficient for the purpose of analysis.

3.4.3.2. Machine Clock

A revolution frequency marker referenced to the first bunch injected into the machine is required. This pulse should have a phase resolution of less than one RF bucket and jitter of less than 10ps RMS.

3.4.3.3. Trigger

A trigger is required to start fast data acquisition. Data may be acquired in association with injection, activation of an insertion device, machine parameter measurements or in conjunction with machine physics experiments. For these reasons the trigger is required to be beam locked. The resolution of this trigger should be much less than the bunch frequency and have the lowest possible jitter. This trigger event may also be used to reset the time stamp data for the BPM electronics and as such it will need to be transmitted to all BPMs simultaneously to ensure synchronization of time stamps. Careful attention will be paid to ensure this trigger reaches each BPM simultaneously and is immune to drift due to temperature.

3.4.3.4. Post mortem event

A trigger is required to capture post mortem data from the BPM electronics. This signal needs to be generated on a fault condition in the machine so that orbit data can be captured leading up to the fault condition. As such this trigger and its timing is common to all BPMs.

In addition to the triggers going to the BPM electronics the current design of the system will provide feedback for machine protection. If the orbit falls outside of

a specified boundary a machine event will be generated to abort the current fill. This signal will be an input to the event receiver and transmitted back to the EVG so that an abort signal can be disseminated to the systems involved in a beam dump.

3.4.4. Vacuum systems

The vacuum system will require accurate time stamp information. Time tamping is covered in section 5.

3.4.5. Beam diagnostics

TBD will discuss this set of requirements at Diamond.

3.4.6. Radio Frequency systems

3.4.6.1. LINAC

The LINAC Gun requires a unique set-up since the hardware is at high potential. An optically isolated trigger needs to be transmitted to the Gun. The requirements for timing include trigger generation as well as injection pattern selection. Triggers for the LINAC in general may be derived from an EVR with a resolution of 10ps and a delay range of greater than 10 seconds. Bunch patterns can be programmed onto the event link with a resolution of 8ns. Further improvement in resolution is then realized in the event receiver by using the CML or PECL outputs.

At this time the requirements of the BNL timing system for the LINAC are not clear. The RFQ refers only to a start injection trigger and the balance of timing may be provided by the vendor. The RF group plans to develop the hardware needed for this system in collaboration with the LINAC vendor the controls group will collaborate on the pattern generation software and integration.

3.4.6.2. Booster

TBD… Standard EVR triggers are available for the use of the RF Group.

3.4.6.3. Storage Ring

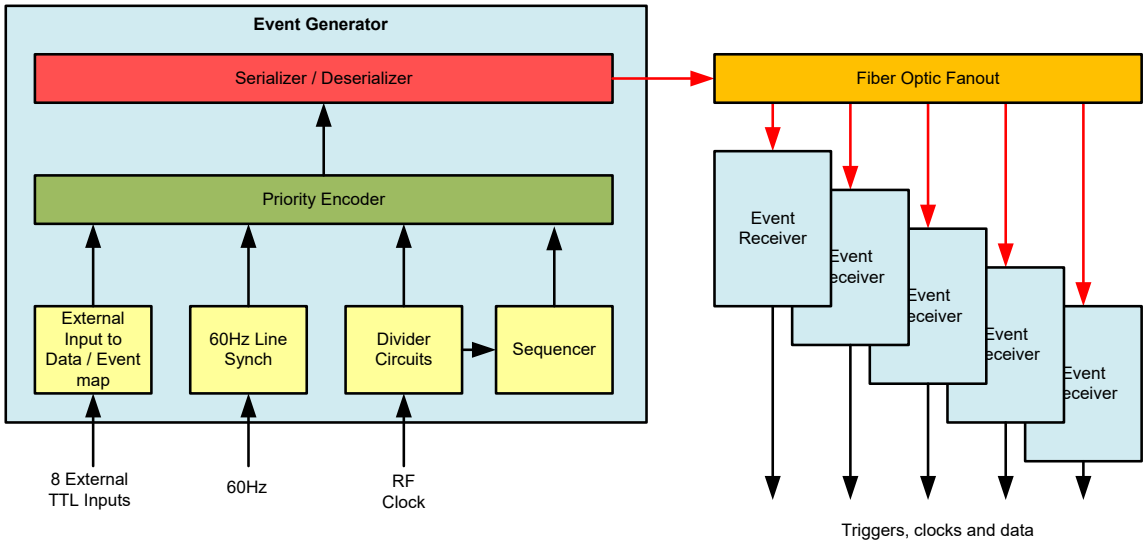
TBD… Standard EVR triggers are available for the use of the RF Group.

3.4.7. Machine protection system

In the event of a fault the timing system can be used to create a global event to abort the beam. A PCI EVR has been developed that will transmit a pulse back to the EVG. The EVG can then generate an abort event a broadcast it to any other EVR in the system. The response time is less than 5us from trigger in to event out.

4. Theory of operation

4.1. Structure of the timing system



Simplified block diagram of the timing system
Figure 3.1

Figure 3.1 shows the topology for the timing system. Events are created in the EVG from several sources. In addition to events the EVG also creates a real time data stream that can be used to carry clocks, data and machine state. Each event and data source is discussed here.

The event link is a data stream transferred over fiber optics to event receivers throughout the complex. The data rate over the fiber is five times the RF frequency or approximately 2.5GHz. Since the event link data rate is derived from the RF clock events are indirectly locked to the 500MHz bunched beam pulses in both the Booster and Storage Ring machines. The fiber is a point to point link therefore it must be fanned out at the source and cabled to the areas where timing is required. Matched length fibers will be used to ensure that the arrival time of events is simultaneous where required. Climate control in the ring

is important to limit drift due to expansion of the fibers. This phenomenon will be examined at the Diamond Light source and mitigated if necessary.

Event receivers will be placed near any equipment that requires timing information. The EVR is capable of producing pulses by decoding the event link data. There are 14 output channels on the VME EVR, 6 on the cPCI EVR and 4 on the PMC version. Each channel is capable of decoding an event and generating a delayed pulse with variable length and polarity. In addition to triggers Time stamps and data are available from the EVR. This chapter will cover these capabilities in detail.

4.2. Event Generator

The Event Generator is designed to create an optical data stream derived from the RF clock. The data stream has system events and an 8 bit data word (distributed bus) encoded onto it and it is updated at the RF frequency divided by four (RF/4). This results in an event time resolution of ~8ns. The encoding scheme is 8b10b and this is done to ensure that there are enough logic state transitions so that the transmit clock can be recovered by the EVRs.

Event Code	Code Name	EVG Function	EVR Function
0x00	Null		
0x70	Seconds ‘0’		Shift 0’s into the time stamp register
0x71	Seconds ‘1’		Shift 1’s into the time stamp register
0x7A	Heartbeat		Reset heartbeat monitor
0x7B	Synchronize prescalers		Synchronize prescalers
0x7C	Increment time stamp		Increment time stamp
0x7D	Reset Timestamp		Reset Timestamp
0x7F	End of Sequence	Stop sequencer	

Table 4.Table 4.1
Reserved events and functions

An event is defined as a time of interest in the machine cycle. For example injections start. Such an event may kick off a process that includes firing the electron gun some number of times to inject beam into the LINAC. Another event might start the Booster dipole ramp generator. Once an event occurs

triggers can be produced in the event receiver with programmable delay and pulse width to turn on or change the state of hardware devices. An event is an 8 bit word therefore there are 256 possible events. Some event codes are reserved so the actual number of useful events is 248. Reserved codes and their functions are shown in table 4.1.

4.2.1. Event Sources

4.2.1.1. Sequencer(s)

The sequencer is a schedule of up to 2048 system events mapped in time. When the current time is equal to the time of the next scheduled event that event is transmitted over the event link for decoding in the EVRs. The sequencer will be used to define the acceleration cycle in the LINAC, Booster and Storage ring. The definition of the acceleration process is controlled by the super cycle generator application.

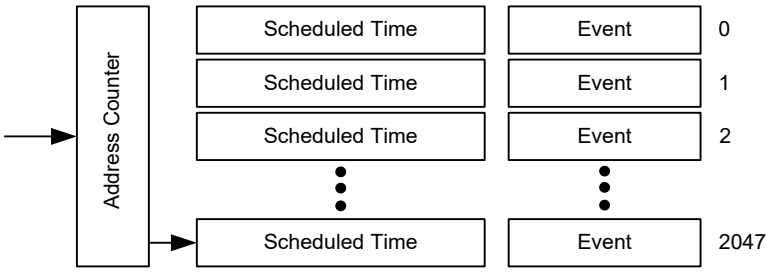


Figure 3.2
Structure of the Sequencer

There are two Sequencers in the EVG of which only one is active*. The second can be updated and a context switch will bring the second sequencer to the foreground. An active sequencer should not be modified (work needs to be done to improve the way updating the sequencer is accomplished see the notes in appendix a).

There are several ways to start the sequencer, external TTL trigger, software trigger, internal counter or from the line synchronous trigger source. Combinations of the methods are also possible.

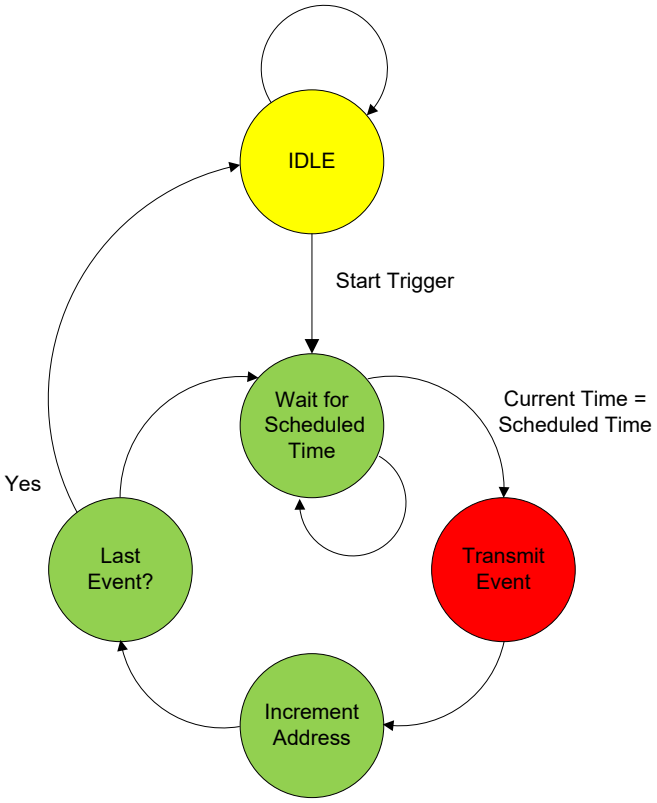


Figure 3.2

Figure 3.2 shows the process followed by the Sequencer. If the sequencer is not running it waits in the idle state until a start event is received. While in the idle state the current time is held at zero. When the trigger comes the current time is incremented and compared to the scheduled time of the event waiting in the event Q. when the time matches the event is transmitted over the event link and the address pointer in the Q is incremented. If this was the last event in the Q the sequencer returns to the idle state and waits to be triggered, if not the sequencer waits for the next scheduled event.

Note: The sequencer can be placed in repeat mode. If repeat is true the sequencer will reset the current time register and address and start the sequence over.

4.2.1.2. External Triggers

There are 8 available inputs to the EVG that may be used as external triggers. These triggers can be mapped to events on the event link and will be transmitted

in the next available time slot on the link (see priority encoder). The input is TTL and is edge sensitive.

4.2.1.3. Multiplexed Counters

There are eight multiplexed counters. These hardware dividers use the RF/4 event clock to produce derivative clock signals for the machine. The divide ratio can be set to values in the range of 2 to 2^{32}-1. For the NSLS-II this means clock frequencies from 62.5MHz down to 29mHz. These clocks can create events, drive bits on the distributed bus, trigger the Sequencer and multiplexed counter 7 can be used in AC line synchronization.

4.2.1.4. AC Line Synchronization

The EVG provides synchronization to the AC line frequency. This frequency can be divided by an 8 bit programmable divider and the output delay can be adjusted from 0 to 25.5ms in 0.1ms steps thus adjusting the phase. After adjusting the frequency and phase the output is re-synchronized to either the event clock or the output of multiplexed counter 7. If no manipulation is required the divider and phase shifter can be bypassed and the line frequency can directly control a trigger event or start the sequencer.

4.2.2. Distributed Bus Data Stream

In addition to the 8 bit event code an eight bit data stream is transmitted for reception in the EVRs. This distributed bus can be used for clock distribution and real time data from the external inputs to the EVG. The distributed bus can also be shared with transmission of a variable length data buffer. This results in two 8ns time slots, one for the distributed bus and one for sending buffer data. Note: this takes away the ability to use the distributed bus for clock generation.

4.2.3. Data Buffer

A data buffer of up to 2k bytes can be broadcast from the EVG to the EVRs. The data is stored in a dual port memory block in the EVG that is directly addressable from the VME bus. Sending the data buffer is triggered by software only. Transfer of this data buffer uses half the bandwidth of the distributed bus.

4.2.4. Time Stamping

Accurate time can be read by the IOC from the EVR. The resolution of the time stamp register is 1us. Time of day information is sent to the timing system once per second and is synchronously updated in all EVRs by the one second event. At the same instant the time of day is updated the microsecond counter is reset. The microsecond register is then an offset to the time of day. The EVG takes four signals from the General Machine Time system: A one pulse per second clock, a 1MHz clock and a serial stream of events that will set the seconds registers in remote EVRs.

4.3. Event Receiver

The event receiver is designed to take in the event link and produce timing triggers, system interrupts, time stamps and to provide data to devices throughout the complex. This section describes each function and section 7 covers the software interface.

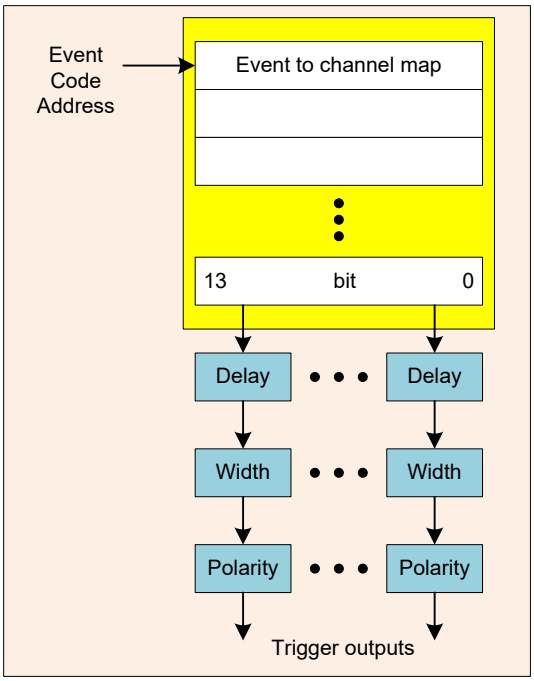


Figure 4.3.1
Event code to channel mapping

System events are mapped to the output ports using the mapping ram (figure 4.3.1). The 8 bit event addresses the ram and bits that are set cause the event to trigger a channel. This allows for multiple events triggering the same device.

4.3.1. Trigger outputs

The event receiver offers up to 14 flexible trigger outputs. Each channel can decode events and create a delayed output with variable pulse width. 10 channels have a maximum delay of approximately 34 Seconds and pulse widths from 8ns to 524us. Four channels offer extended delays and pulse widths of up to 625 hours.

4.3.1.1. TTL Outputs

Eight outputs are available on the front panel of the EVR (four on the front panel of the RF version of the EVR). Each output will drive TTL levels into 50 Ohms and can be mapped to a programmable delay and width pulse, a prescaler, Toggle bit or distributed bus bit by setting the appropriate mapping register bit (see software section). The output may also be held high or low.

4.3.1.2. CML Outputs

CML (Current Mode Logic) outputs are available on the RF version of the EVR. There are 3 CML outputs that offer high frequency low jitter differential signals to devices which have more stringent timing requirements. CML outputs are programmed in the same way as TTL outputs however there is additional logic which allows these channels to output frequencies up to the RF reference clock frequency.

Figure 4.3.1.2 shows an example of how to use the added logic on the CML output to create a clock signal at 500MHz. The pattern generator (yellow) holds a sequence of bits that are output serially. There are four pattern generators one for each possible transitional state of the input

pulse from the mapping register (parallel load in the diagram). The example shows the pattern for a 0 -> 1 transition of the input. There are patterns for the following transitions:

0 -> 0
1 -> 0
1 -> 1

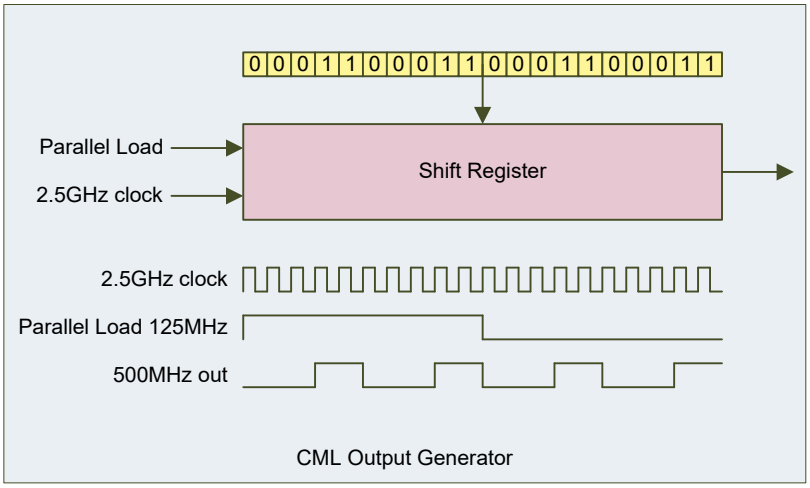


Figure 4.3.1.2
Example 500MHz CML output using a pattern generator

4.3.1.3. PECL

On the front panel of the EVR and also on the rear transition module there are places to add Universal IO modules. One variety of UIO has provision for supplying differential positive ECL with enhanced delay control. This device has a resolution of 10ps and jitter of less than 5ps.

(Add info here after Jukka's visit)

4.3.2. Prescaler Outputs

There are three prescalers that can be routed to the front panel outputs. These prescalers divide the event clock and can be used to produce system clocks. Event 0x7B synchronously resets the prescalers in all EVR's so that the phase of these signals can be the same throughout the machine.

4.3.3. Latched Level Outputs

There are 7 outputs available on the rear transition module for latched TTL signals. Each latch channel has set and reset inputs associated with the event map. Starting with map bit 0 as the reset to latch (LVL) 0 and bit 1 for the set each latch can be associated with two events. The reset bit for latch n is 2n and the set bit is 2n+1. Each latch can be enabled or disabled using the output level enable register.

4.3.4. Distributed bus

The distributed bus described in section 4.2.2 can be an output of the EVR. The outputs can be sent the rear transition module or the front panel outputs. The rear transition module shares these pins with programmable delay outputs. The software section describes how to configure these pins.

The distributed bus is shared with the transmission of a data buffer. This is time division multiplexed and impacts the ability to use the distributed bus for clock signals.

4.3.5. Data Buffer

A variable size data buffer can be broadcast from the EVG and, if enabled received in the EVR. The maximum length is 2k bytes. Upon reception of a data buffer the EVR can generate an interrupt. The interrupt handler routine for the EVR can then read the data buffer status register to see if the data buffer caused the interrupt and the data can be read by the IOC. Along with the data buffer a checksum is sent. The status of the checksum can also be read from the status register.

4.3.6. IOC Interrupts

There are six interrupt sources on the EVR:

Event link receive failure
Lost heartbeat
New event in the event FIFO
Event FIFO full

Data buffer receive complete
Delay channel interrupt

The interrupt priority is set on initialization as is the interrupt vector. The heartbeat will be monitored as a fault indicator for the system. The delay channel interrupt gives the capability of having an event cause a local interrupt at any time after the occurrence of the event.

4.3.7. Time stamping

Local time stamps are referenced back to the GPS disciplined time server there are two registers for the EVR time stamp: a seconds register which is updated by the time server and a microseconds register which acts as an offset to the seconds register. The time stamp register is updated by the one pulse per second event, at this instant the current time preloaded from the time server is transferred to the seconds register and the microseconds register is reset to zero (figure 4.3.7.1).

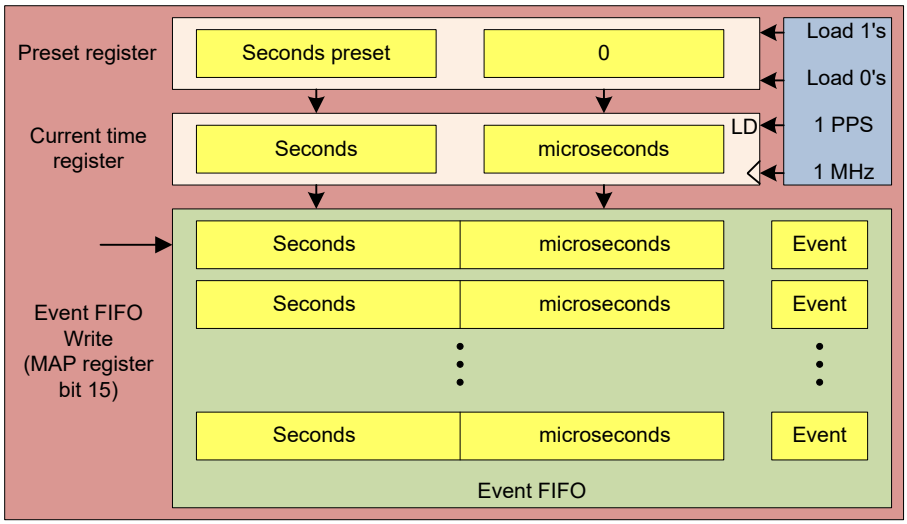


Figure 4.3.7.1
Time stamping structure

The Event FIFO may store any event received by the EVR and store the time at which it occurred. The Event FIFO can store up to 511 event / time stamp pairs. Both the current time register and the Event FIFO can be read by the IOC. System wide this is the most accurate time available. Through the timing system the

registers across all EVRs are updated simultaneously. This allows for precise time reconstruction during post mortem analysis.

4.3.8. Local Event trigger input

TTL In 0 on the front panel may be used to cause a local event to occur in that EVR. This trigger will cause the event stored in the “external event code register” to be sent to the mapping ram. All downstream functionality is the same as an event over the event link.

4.3.9. Interlock Input

All front panel and rear transition module outputs can be disabled on the occurrence of an interlock signal. Each channel has independent control of whether or not it responds to the interlock signal. For this feature to be available a universal IO module (UNIV-TTLIN-IL) must be installed in rear transition module slot 0/1.

4.4. General Machine Time

GMT will be provided by a Symmetricom frequency and time standard. This device is GPS disciplined and has a Rubidium oscillator for accurate time and frequency generation over an extended time. The output from this device is the time of day and a 1 Pulse Per Second clock tick that is used to update the time stamp registers in all EVRs in the system.

This device also acts as a network time server and can be used to synchronize applications across the accelerator complex.



Xli GPS disciplined time and frequency source

The Rubidium frequency standard has a phase noise of -130dBc at 10Hz offset from the carrier. This specification makes the device appropriate for use as the 10MHz frequency standard used by the RF Oscillator.



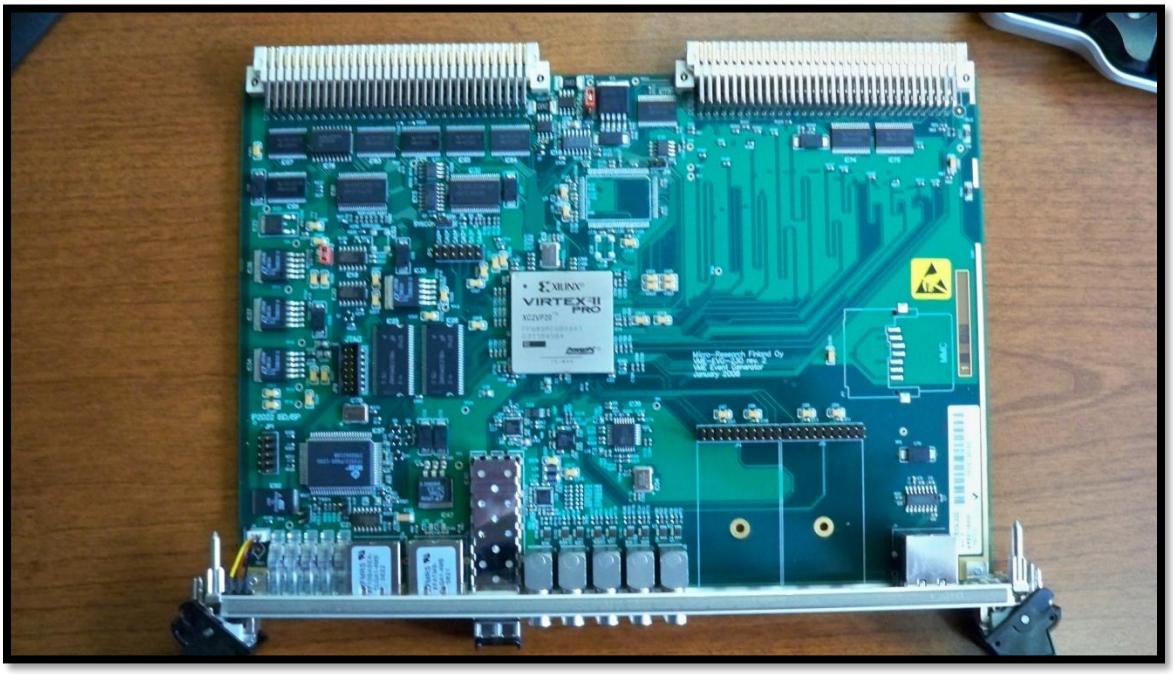
Stanford Telecom Rubidium oscillator

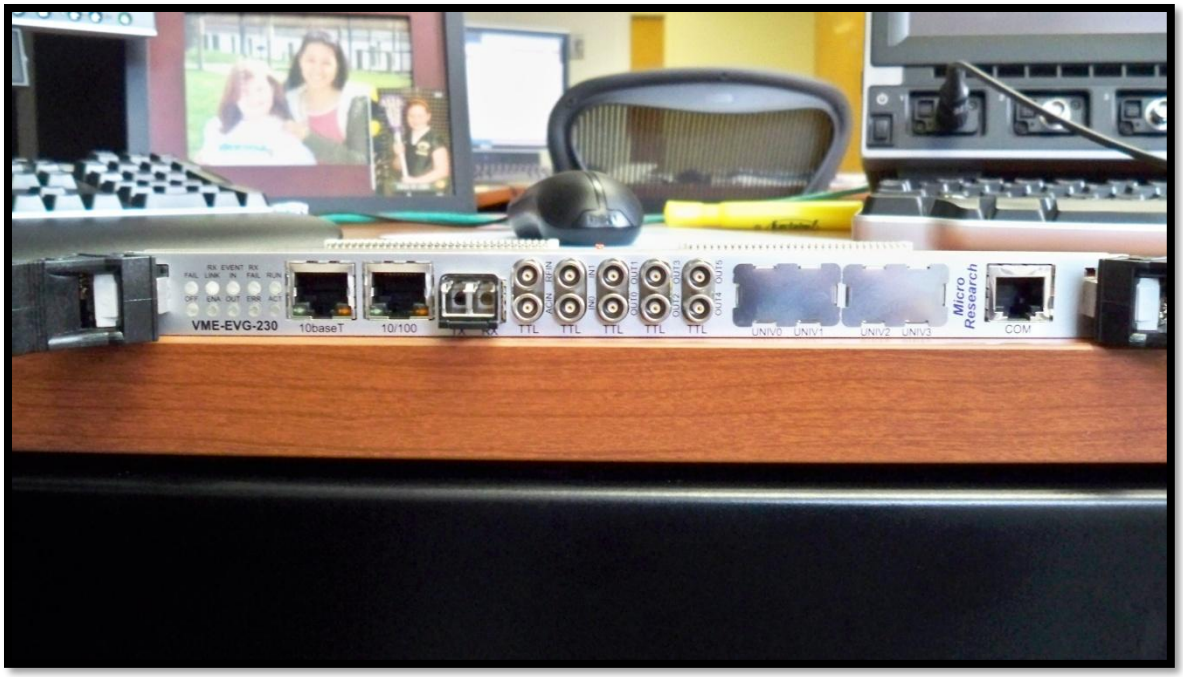
Time of day information is read from the Xli through the communications port and translated to a series of event codes used by the EVR to update the time stamp register (section 4.3.7). The 1PPS output is used to generate event code 0x7D which loads the current time into the EVR seconds register.

5. System Components

This section outlines the hardware used to realize the timing system in NSLS-II. Hardware components such as Event Generators and Receivers, fiber optic fan-out modules and LINAC timing components will be purchased from Micro Research Finland. Electrical fan-out, GMT interface module and Scope multiplexers will be designed in house.

VME EVG 230



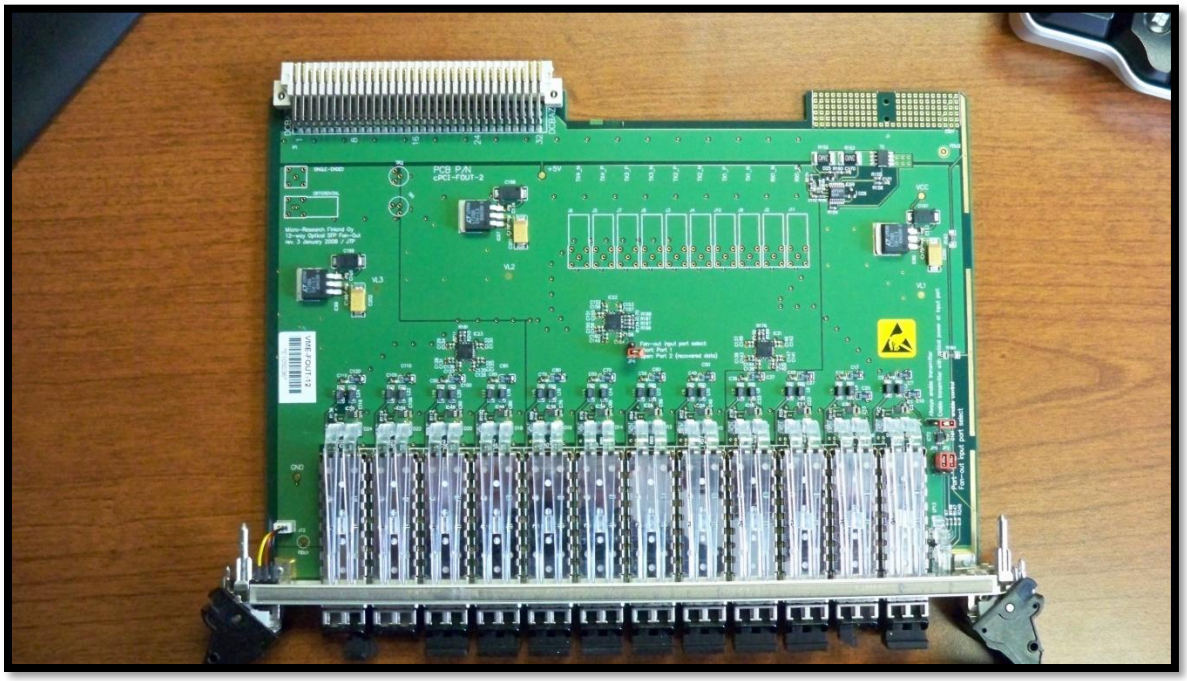

VME-EVG-230
10baseT
10/100
TTL
TTL
TTL
TTL
TTL
UNIV0
UNIV1
UNIV2
UNIV3
Micro Research
COM

VME EVR 230


XILINX
VIRTEX-II PRO
JTAG
UNIV-TTL



1:11 Optical Fan Out FOUT12



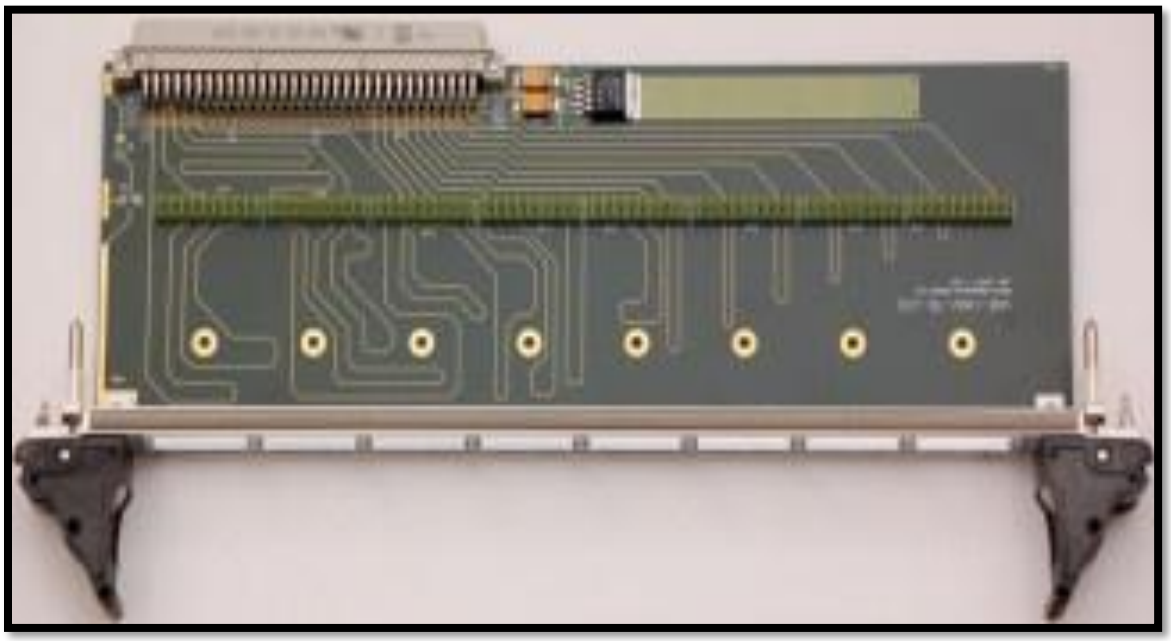


PMC EVR





Universal Transition module



6. Infrastructure and Layout

7. Software

7.1. Event Generator

7.1.1. Sequencer Control

Trigger set up

There are four possible sources for a Sequencer trigger: a multiplexed Counter, the AC line, a software trigger, or if a TTL pulse is received on the rear transition module trigger input at Universal I/O 9.

To trigger the sequencer from the AC line input set the following field:

In the .db file: *field(ASQN, "YES")*

Or from the command prompt: *caput <eg:record:name>.ASQN YES*

*Where N is 1 or 2 depending on the sequencer being modified

To trigger sequencer 1 from multiplexed counter 0 the following field must be set:

In the .db file: *field(MSQ1, "YES")*

Or from the command prompt: *caput<eg:record:name>.MSQ1 YES*

To trigger sequencer 2 from multiplexed counter 1 the following field must be set:

In the .db file: *field(MSQ2, "YES")*

Or from the command prompt: *caput<eg:record:name>.MSQ2 YES*

Sequencer Event setup

Once you have determined how you want the sequencer to trigger, you must add events with scheduled times to the queue. Each event in the sequencer is an egevent record.

```
record(egevent, "sequencer:1:event:1") {
        # Initialization
        field(DTYP, "MRF Event Generator")

        # Set what EVG this event maps to
        field(OUT, "C0 S0 @")

        # Event number to be sent out
        field(ENM, "0x01")

        # Delay time from the triggering of the sequencer
        field(DELY, "20")

        # Unit of measurement for delay time
        field(UNIT, "Nanoseconds")
}
```

Egevent records are processed in the order that they appear in the .db file. If two events have the same delay time then the one that gets processed first will get the first priority. The second event will be sent out one clock cycle later.

7.1.2. External Triggers

Set up trigger source

To use an external trigger, you must specify one of the following four sources: a multiplexed counter, the AC line sync, a front panel TTL input or a TTL input at Universal I/O 0 through 7 on the rear transition module.

Multiplexed counters must be enabled before use. **Remember**: Multiplexed counters map directly to the external trigger of the same number e.g. multiplex counter 0 will control external trigger event 0 if enabled. Set the following field to enable a multiplexed counter:

In the .db file: *field(MENE, "YES")*

Or from the command prompt: *caput <eg:record:name>.MENE YES*

*Where N is 0 thru 7 depending on what multiplexed counter you want to use

The AC line sync only controls external trigger event 0, which can be enabled by setting the following field:

In the .db file: *field(AEV0, "YES")*

Or from the command prompt: *caput <eg:record:name>.AEV0 YES*

There are two TTL inputs on the front panel of the EVG. Each TTL input can be set to trigger any of the eight external trigger events. To do so set the following field:

In the .db file: *field(TNDX, "YES")*

Or from the command prompt: *caput <eg:record:name>.TNDX YES*

*Where N is either 0 or 1 depending on what TTL input you are using and X is 0 thru 7 depending on what multiplexed counter you want the TTL input to control

The rear transition module allows Universal I/O 0 through 7 to trigger an external trigger event by default (the software is not needed). **Remember** that Universal I/O 0 through 7 maps directly to the external trigger event of the same number e.g. Universal I/O 0 will control external trigger event 0.

Set up the external trigger event

Before an external trigger event can be set up you need to determine what trigger event you will be using and what event code it will be sending out.

To enable an external trigger event you must set the following field:

In the .db file: *field(ETEN, "YES")*

Or from the command prompt: *caput <eg:record:name>.ETEN YES*

*Where N is 0 thru 7 depending on what external trigger event you want to use

To set the event code that is sent out by an external trigger event you must set the following field:

In the .db file: *field(ETN, "0xNUM")*

Or from the command prompt: *caput <eg:record:name>.ETN 0xNUM*

*Where N is 0 thru 7 depending on what external trigger event you are using and NUM is a hex value currently not being used

7.1.3. Multiplexed Counters

Set the prescaler

To set the prescaler for a Multiplexed counter you must set the following field:

In the .db file: *field(MCNP, "NUM")*

Or from the command prompt: *caput <eg:record:name>.MCNP NUM*

*Where N is 0 thru 7 depending on what multiplexed counter you are going to use and NUM is the value of the prescaler which can be from 2 to 2^{32}-1

Determine what you want the multiplexed counter to do

Multiplexed counters can be used to drive bits to the distributed bus, send out event codes, trigger either sequencer, or be used in relation to the AC line sync.

When using a multiplexed counter to drive a bit into the distributed bus you must **remember** that each multiplexed counter is directly mapped to the distributed bus bit of the same number e.g. multiplexed counter 0 is mapped to distributed bus bit 0. To enable a multiplexed counter to drive a distributed bus bit you must set the following field:

In the .db file: *field(MDNE, "YES")*

Or from the command prompt: *caput <eg:record:name>.MDNE YES*

*Where N is 0 thru 7 depending on what multiplexed counter you want to use

Using the multiplexed counter to send out an event is the same as described in section 7.1.2. **Remember**: Multiplexed counters map directly to the external trigger of the same number e.g. multiplexed counter 0 will control external trigger event 0. In order to enable a multiplexed counter set the following field:

In the .db file: *field(MENE, "YES")*

Or from the command prompt: *caput <eg:record:name>.MENE YES*

*Where N is 0 thru 7 depending on what multiplexed counter you want to use

Sequencer 1 can be triggered by multiplexed counter 0 by setting the following field:

In the .db file:	*field(MSQ1, "YES")*
Or from the command prompt:	*caput <eg:record:name>.MSQ1 YES*

Sequencer 2 can be triggered by multiplexed counter 1 by setting the following field:

In the .db file:	*field(MSQ2, "YES")*
Or from the command prompt:	*caput <eg:record:name>.MSQ2 YES*

An AC line sync may be synchronized to multiplexed counter 7. **We do not understand the purpose of this**, but if you wish this enabled set the following field:

In the .db file:	*field(ASYN, "Mux Counter 7")*
Or from the command prompt:	*caput <eg:record:name>.ASYN "Mux Counter 7"*

7.1.4. AC Line Synchronization

Determine what you want the AC line sync to do

The AC line sync can be used to trigger either of the sequencers or to enable external trigger event 0.

To trigger a sequencer from the AC line sync set the following field:

In the .db file:	*field(ASQN, "YES")*
Or from the command prompt:	*caput <eg:record:name>.ASQN YES*

*Where N is 1 or 2 depending on the sequencer being modified

To use the AC line sync to enable external trigger event 0 set the following field:

In the .db file:	*field(AEV0, "YES")*
Or from the command prompt:	*caput <eg:record:name>.AEV0 YES*

Optional features

The AC line sync has a few features that are not required, but could be useful depending on your setup. The AC line has a prescaler and it can also be delayed. You may then re-synchronize the AC line to the event clock or multiplexed counter 7 after you have set the prescaler and/or delay. Again, all of this is optional and can be easily bypassed.

To use the AC line sync prescaler set the following field:

In the .db file: *field(ADIV, "NUM")*

Or from the command prompt: *caput <eg:record:name>.ADIV NUM*

*Where NUM can be 1 thru 256

To delay (AKA phase shift) the AC line sync set the following field:

In the .db file: *field(APHS, "NUM")*

Or from the command prompt: *caput <eg:record:name>.APHS NUM*

*Where NUM can be 0 thru 25.5ms and can be in increments of .1ms only

To resynchronize with either the event clock or multiplexed counter 7 set the following field:

In the .db file: *field(ASYN, "TYPE")*

Or from the command prompt: *caput <eg:record:name>.ASYN "TYPE"*

*Where TYPE is either "Event Clock" or "Mux Counter 7"

If you choose to bypass all of this and use AC line sync directly then set the following field:

In the .db file: *field(ABYP, "YES")*

Or from the command prompt: *caput <eg:record:name>.ABYP YES*

7.1.5. Distributed Bus Data Stream

Sending a clock

You may distribute a clock signal from one of two sources: a multiplexed counter or a TTL input.

To use a multiplexed counter to distribute a clock signal set the following field:

In the .db file: *field(MDNE, "YES")*

Or from the command prompt: *caput <eg:record:name>.MDNE YES*

*Where N is 0 thru 7 depending on what multiplexed counter you wish to use

To use a TTL input to distribute a clock signal set the following field:

In the .db file: *field(TNDX, "YES")*

Or from the command prompt: *caput <eg:record:name>.TNDX YES*

*Where N is 0 or 1 depending on what TTL input you are using and X is 0 thru 7 depending on what distributed bus bit you want the TTL input to set

7.1.6. Data Buffer

Set up

A couple of things must be set up in the event generator before you can trigger a transmission of data from the data buffer to multiple event receivers. You must enable the data buffer and set its size according to the amount of data you have put into it.

Enabling an EVG to use the data buffer means that bandwidth will be shared between it and the distributed bus. To enable the data buffer (and subsequent sharing of bandwidth) set the following field:

In the .db file: *field(DBEN, "Enabled")*

Or from the command prompt: *caput <eg:record:name>.DBEN Enabled*

The size of the data buffer is set with the following field:

In the .db file: *field(DBSZ, "VAL")*

Or from the command prompt: *caput <eg:record:name>.DBSZ VAL*

*Where VAL can be from 4 to 2048 (bytes) and can only be in multiples of 4 bytes

Transmit data

Before you decide that you want the data to be transmitted you **MUST** make sure your EVRs are configured to receive the data buffer. See **section 7.2.5** for more information on setting up an event receiver to receive data.

To transmit the data set the following field:

From the command prompt (only): *caput <eg:record:name>.DTRG 1*

*Please note that the value does not have to be '1' but can be any value that is not equal to zero

7.1.7. Time Stamping

Send out time stamping event codes

The only part that an EVG plays in time stamping is to send out predefined event codes. These can either shift 1s or 0s into the seconds value register or latch the time stamp.

The seconds counter contains the number of seconds since the EPICS epoch. It is a "read-only" field that is indirectly initialized by sending out event codes 0x70 and 0x71 which will shift in a '0' or a '1' respectively as the least significant bit.

To shift a '1' into the register as the least significant bit send out event code 0x71:

From the command prompt: *caput <eg:record:name>.VME 113*

*Please note that the event code value must be in decimal and not in hexadecimal. This is true whenever sending out a single software event

To shift a '0' into the register as the least significant bit send out event code 0x70:

From the command prompt: *caput <eg:record:name>.VME 112*

After this is done you will want to latch the seconds register, which will cause the current time to be equal to the EPICS epoch plus the seconds register. To latch the timestamp send out the event code 0x7D:

From the command prompt: *caput <eg:record:name>.VME 125*

7.2. Event Receiver

Mapping RAM

Each event receiver will be programmed to respond to certain events. The mapping RAM contains all of this functionality and is controlled by a single EPICS record type: erevent. Each erevent record contains the event code that it will handle and the MAP bit(s) that it will drive. Here is an example of an erevent record:

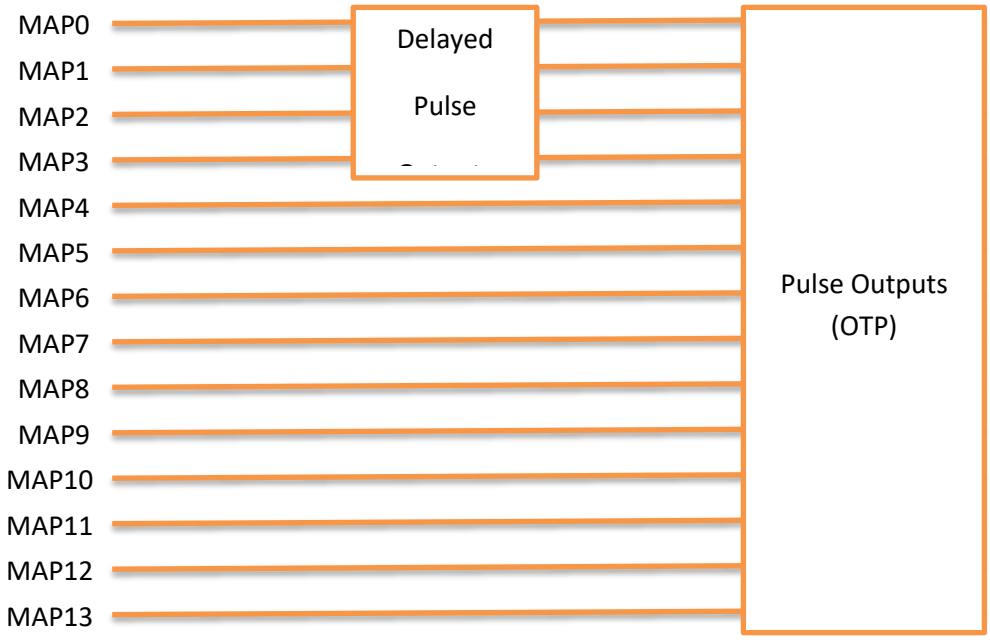
```
record(erevent, “mapping:ram:event:1”) {
        # Inititalization
        field(DTYP, “MRF Event Receiver”)
        field(ENAB, “Enabled”)

        # Set what EVR this event maps to
        field(OUT, “#C0 S0 @”)

        # Set what event code this record maps to
        field(ENM, “0x01”)

        # Set the RAM (MAP) bit
        field(OUT0, “Enabled”)
}
```

This record indicates that the first event receiver in your crate is programmed to respond to event code 0x01 by enabling the MAP0 (OUT0 in EPICS) bit. The bit that is enabled will correspond directly to which pulse output the event receiver is able to send out. Here is a diagram showing each MAP bit as it relates to the pulse output it may drive:



It is also worth noting that the MAP bits may drive seven flip-flop outputs. We have not yet experimented with these.

7.2.1. Trigger outputs

Front panel output mapping

The front panel of the event receiver has four TTL outputs, six CML outputs and the possibility of adding up to four Universal outputs. The signal output on one of these front panel outputs is set in the following manner:

For either a TTL or CML output:

In the .db file: *field(FPSN, "VAL")*

Or from the command prompt: *caput <er:record:name>.FPSN VAL*

*Where N is 0 through 6 depending on what front panel output you are using

For a Universal output:

In the .db file: *field(UNVN, "VAL")*

Or from the command prompt: *caput <er:record:name>.UNVN VAL*

*Where N is 0 through 3 depending on what universal output you are using

In both cases, the VAL is the signal you wish to read. The following table states which signal each VAL corresponds to:

VAL	Signal
0	Delayed pulse output 0
1	Delayed pulse output 1
2	Delayed pulse output 2
3	Delayed pulse output 3
4	External trigger event 0
5	External trigger event 1
6	External trigger event 2
7	External trigger event 3
8	External trigger event 4
9	External trigger event 5
10	External trigger event 6
11	Pulse output 0
12	Pulse output 1

13	Pulse output 2
14	Pulse output 3
15	Pulse output 4
16	Pulse output 5
17	Pulse output 6
18	Pulse output 7
19	Pulse output 8
20	Pulse output 9
21	Pulse output 10
22	Pulse output 11
23	Pulse output 12
24	Pulse output 13
25	Level output 0
26	Level output 1
27	Level output 2
28	Level output 3
29	Level output 4

30	Level output 5
31	Level output 6
32	Distributed bus data 0
33	Distributed bus data 1
34	Distributed bus data 2
35	Distributed bus data 3
36	Distributed bus data 4
37	Distributed bus data 5
38	Distributed bus data 6
39	Distributed bus data 7
40	Prescaler 0
41	Prescaler 1
42	Prescaler 2
62	Output tied high
63	Output tied low

TTL outputs

There are two types of pulses that can be output via TTL: a normal pulse and a delayed pulse. You may also toggle between using a normal pulse and using the distributed bus.

Pulse outputs

Pulse outputs are called OTPs for "historical" reasons. Each pulse output has a programmable delay, width and polarity.

To enable a pulse output set the following field:

In the .db file: *field(OTPN, "Enabled")*

Or from the command prompt: *caput <er:record:name>.OTPN Enabled*

*Where N is 0 through the hexadecimal value D depending on what pulse output you want to use

To set the delay of a pulse output set the following field:

In the .db file: *field(OTND, "VAL")*

Or from the command prompt: *caput <er:record:name>.OTND VAL*

*Where N is 0 through the hexadecimal value D depending on what pulse output you wish to use and VAL can be from 0 to 32^2-1 clock cycles

To set the width of a pulse output set the following field:

In the .db file: *field(OTNW, "VAL")*

Or from the command prompt: *caput <er:record:name>.OTNW VAL*

*Where N is 0 through the hexadecimal value D depending on what pulse output you want to use and VAL is the number of clock periods

To set the polarity of a pulse output set the following field:

In the .db file: *field(OTNP, "TYPE")*

Or from the command prompt: *caput <er:record:name>.OTNP TYPE*

*Where N is 0 through the hexadecimal value D depending on what pulse output you want to use and TYPE is either "Normal" or "Inverted"

If you don't wish to use a pulse output and want to enable the distributed bus signal set the following field:

In the .db file: *field(OTNB, "Enabled")*

Or from the command prompt: *caput <er:record:name>.OTNB Enabled*

*Where N is 0 through 7 depending on what distributed bus data signal you wish you use

Delayed pulse outputs

The only difference between delayed pulse outputs (called DGPs for "historical" reasons) and pulse outputs is that delayed pulse outputs have a prescaler. This allows you to scale down the local clock that is sending this pulse out.

To enable a delayed pulse output set the following field:

In the .db file: *field(DGNE, "Enabled")*

Or from the command prompt: *caput <er:record:name>.DGNE Enabled*

*Where N is 0 through 3 depending on what delayed pulse output you wish to use

The delay of a delayed pulse output is set with the following field:

In the .db file: *field(DGND, "VAL")*

Or from the command prompt: *caput <er:record:name>.DGND VAL*

*Where N is 0 through 3 depending on what delayed pulse output you are using and VAL is from 0 to 32^2-1 clock cycles

The width of a delayed pulse output is set with the following field:

In the .db file: *field(DGNW, "VAL")*

Or from the command prompt: *caput <er:record:name>.DGNW VAL*

*Where N is 0 through 3 depending on what delayed pulse output you are using and VAL is the number of clock periods

The polarity of a delayed pulse output is set with the following field:

In the .db file: *field(DGNP, "TYPE")*

Or from the command prompt: *caput <er:record:name>.DGNP TYPE*

*Where N is 0 through 3 depending on what delayed pulse output you are using and TYPE is either "Normal" or "Inverted"

To set the prescaler of the delayed pulse output set the following field:

In the .db file: *field(DGNC, "VAL")*

Or from the command prompt: *caput <er:record:name>.DGNC VAL*

*Where N is 0 through 3 depending on what delayed pulse output you are using and VAL is 1 through 65535

CML Outputs

CML outputs have the same features as pulse outputs with the addition of extra logic. A special bit pattern may be given for the current state of the pulse. The current state can be the falling state (1 -> 0), the high state (1 -> 1), the low state (0 ->0), or the rising state (0 -> 1).

To enable the CML output you must send a specific control word to set the correct bits in the CML control register. The following field and control word will do so:

In the .db file: *field(CMNE, "0x01")*

Or from the command prompt: *caput <er:record:name>.CMNE 0x01*

*Where N is 4 through 6 depending on what front panel CML output you plan on using

To specify the bit pattern for the falling state set the following field:

In the .db file: *field(CMNF, "PAT")*

Or from the command prompt: *caput <er:record:name>.CMNF PAT*

*Where N is 4 through 6 depending on what front panel CML output you plan on using and PAT is a 20 bit pattern in hexadecimal form

To specify the bit pattern for the high state set the following field:

In the .db file: *field(CMNH, "PAT")*

Or from the command prompt: *caput <er:record:name>.CMNH PAT*

*Where N is 4 through 6 depending on what front panel CML output you plan on using and PAT is a 20 bit pattern in hexadecimal form

To specify the bit pattern for the low state set the following field:

In the .db file: *field(CMNL, "PAT")*

Or from the command prompt: *caput <er:record:name>.CMNL PAT*

*Where N is 4 through 6 depending on what front panel CML output you plan on using and PAT is a 20 bit pattern in hexadecimal form

To specify the bit pattern for the rising state set the following field:

In the .db file: *field(CMNR, "PAT")*

Or from the command prompt: *caput <er:record:name>.CMNR PAT*

*Where N is 4 through 6 depending on what front panel CML output you plan on using and PAT is a 20 bit pattern in hexadecimal form

7.2.2. Prescaler Outputs

There is currently no EPICS support for setting the prescaler values.

7.2.3. Latched Level Outputs

Mapping RAM

Not only can event codes be mapped to pulse outputs via the mapping RAM, but they also can map to the set/reset capabilities of a latched level output. There are seven of these latched level outputs available (numbered 0 through 6) and depending on what MAP bit is set will determine what condition the latched level output is in. Setting MAP(2X) will reset latched level X and setting

MAP(2X+1) will set latched level X e.g. enabling MAP2 (remember it is OUT2 in the EPICS erevent record) will set latched level 1.

Enabling

Once you have an event code that is mapped to the correct bit(s) then you will want to enable the latched level output in the event receiver. This is done by setting the following field:

In the .db file:	*field(OTLN, "Enabled")*
Or from the command prompt:	*caput <er:record:name>.OTLN Enabled*

*Where N is 0 through 6 depending on what latched level output you wish to use

7.2.4. Distributed bus

The only thing that needs to be set in an event receiver, as it pertains to the distributed bus, is the correct mapping of the front panel output or the universal output being used to read the distributed bus data. For more details on this see the chart in **section 7.2.1.**

7.2.5. Data Buffer

Set up

For an event receiver to be able to receive data from an event generator you must enable data buffer reception by setting the following field:

In the .db file:	*field(DBEN, "Enabled")*
Or from the command prompt:	*caput <er:record:name>.DBEN Enabled*

Interrupt

Once an event receiver has received some piece(s) of data from an event generator you will want to trigger an interrupt. This will allow you to run an interrupt service routine (ISR) on the data you have just obtained. To enable a data buffer interrupt on an event receiver set the following field:

In the .db file:	*field(DINT, "Enabled")*
Or from the command prompt:	*caput <er:record:name>.DINT Enabled*

From this point you will want to modify the ISR to handle the data in the way you want. More information on interrupts and ISRs are in the **section 7.2.6**.

7.2.6. IOC Interrupts

General

The device support we currently have has an interrupt service routine already written. The only thing the user needs to do is make sure each type of interrupt is handled to their specifications. The ISR is called **ErIrqHander()** and is located in the file **drvMrfEr.c**. This routine will check all of the interrupt conditions to determine what type has happened. Each condition has its own "listener" function that will get called. The listener function is created by the user and is used to determine how each interrupt is handled. Examples of possible listeners will be shown in the following sections on each interrupt type.

Data buffer interrupt

If you have the data buffer interrupt enabled then the event receiver will fire off an interrupt whenever a data buffer transmission is received. The listener function that you create must be registered during initialization in **ErInitRecord()**. This is done by calling the function **ErRegisterDBuffHandler()** with the function you have created as a parameter. An example function is as follows, which will output the first element (a 32 bit integer in this example) in the data buffer:

```
LOCAL_RTN
void ErDevDBuffFunc(ErCardStruct *pCard, epicsUInt16 BuffSize, void *DataBuffer)
{
        // Message to be output
        char msg[256]

        // Cast void pointer as 32 bit integer pointer
        epicsUInt32 * data = DataBuffer;

        // Create and send your message
        epicsSnprintf(msg, EVR_INT_MSG_LEN, "Integer value: %u\n", data[0]);
        epicsInterruptContextMessage(msg);
}
```

7.2.7. Time stamping

The EVR has an event FIFO which keeps track of events and their time stamps. We need to do more experimenting on the event receiver side as it pertains to time stamping.

7.2.8. Local Event trigger input

A local event trigger will cause an event receiver to read the external event code register which will be handled by the mapping RAM. On initialization the device support sets this event code to 0x76. I'm not sure what the purpose is of this and there currently is no EPICS support to modify this field.

7.2.9. Interlock Input

There is no software related information for this feature.

8. Software Development

8.1. RTEMS

Installation

The device support application that we are currently using can be downloaded from the following SourceForge page:
http://epics.svn.sourceforge.net/viewvc/epics/applications/trunk/mrfEventSystem/

You will notice that the application has a README file that does a relatively decent job of explaining the installation process. It takes a rather general approach to installation and you may, depending on your set up, run into a few issues. Here are some issues we had and how we were able to fix them:

- After unpacking the software (step 2 in the README) make sure you cd into the mrfEventSystem directory. Otherwise you will be running the makeBaseApp script in the wrong location.
- After editing MRF_CONFIG_SITE_<ioc-arch> (step 5 in the README) make sure you do the following:
    - Configure/CONFIG_SITE
        - Uncomment CROSS_COMPILER_TARGET_ARCHS and enter the correct parameter (it will involve RTEMS and the ioc you are using)

    - Configure/RELEASE
        - Check to make sure that the EPICS_BASE parameter is the correct pathway
- After running 'make' you may run into some of the following issues:
    - basicIoOps.h is missing (http://www.slac.stanford.edu/grp/lcls/controls/global/standards/software/basicIoOps.h)
        - Put the file in /mrfApp/src
    - "Errno" undeclared
        - Add the following header to the file that has the error:
            - #include <errno.h>
    - "ER" redefined
        - Edit mrfApp/src/drvMrfEr.c
            - change the name of the ER () routine to something along the lines of DIAG ()
                - ~ line 5457
            - Change the display routine to run DIAG () instead of ER ()
                - ~ line 6120
    - Object of type 'rtems_irq_connect_data' does not contain member 'next_handler'
        - Take a look at /rtems/rtems-4.9.2/powerpc-rtems4.9.2/mvme3100/lib/include/bsp/irq.h
            - Make sure the following include is near the top of the file
                - #include <rtems/irq.h>
            - Make sure the following include is near the bottom of the file
                - #include <bsp/irq_supp.h>
        - Make the following edits on /epics/timing/mrfEventSystem/mrfApp/src/os/RTEMS/mrfPmcOSD.h
            - Comment out the following includes:
                - #include <rtems.h>
                - #include <rtems/pci.h>
                - #include <rtems/irq.h>
            - Replace any instance of 'pci_read_config_long' with 'pci_read_config_dword'

- Replace any instance of ‘pci_write_config_long’ with ‘pci_write_config_dword’

- Make the following edits on /epics/timing/mrfEventSystem/mrfApp/src/mrfPmc.c
  - Comment out the following lines:
    - printf ("Byte Order is %x\n", _BYTE_ORDER); // line 474
    - printf ("Big Endian is %x\n", _BIG_ENDIAN); // line 475
    - printf ("Little Endian is %x\n", _LITTLE_ENDIAN); // line 476

RTEMS conflict resolution

Once the application is installed you will want to compile it. You will notice that it produces object code files and library files instead of an RTEMS boot file. This is the app was created with little thought for RTEMS in mind, but it is easy to get this application working if you are using RTEMS as your embedded OS.

The first thing you must do is create a new directory that will contain the base application. Create the directory and run the makeBaseApp script to create an IOC application. We will use this base application to link the library files into an RTEMS boot file. The following lines need to be added to the makefile that is located in the xxxApp/src directory of the application you have just created (please note that the file structure used in this makefile may not be the same as the file structure you are using):

```
# Grab the include files from the main application
USER_INCLUDE_RTEMS += -I ../../../../mrfTS/include

# Name of the application you are making
PROD_IOC_RTEMS += mrf

# Create the dbd file from those that have been made in the main application
DBD += mrf.dbd

# .cpp file to derive from mrf.dbd
mrf_SRCS += mrf_registerRecordDeviceDriver.cpp

# The location of the library files the main application has made
```

```
PROD_LDFLAGS_RTEMS += -L /home/XXX/epics/timing/mrfTS/lib/RTEMS-
mvme3100/ -02

# The library files you want to make the boot file from
PROD_SYS_LIBS_RTEMS += mrfCommon mrfEg mrfEr
```

Make sure you compile the main application before you compile this sample application otherwise there will be no library files to load. Once everything is compiled you will have an RTEMS boot file available in the /bin/ directory of the base application.

Startup script

The startup script contains all of the necessary routines to configure EPICS. It will load the necessary .dbd and .db files and call special routines for each piece of hardware to be correctly initialized. Our timing system application also has routines for debugging purposes. Here is a sample startup script:

```
## Example RTEMS startup script

## You may have to change mrf to something else
## everywhere it appears in this file

#< envPaths

## Register all support components
dbLoadDatabase "dbd/mrf.dbd"
mrf_registerRecordDeviceDriver(pdbbase)

## Load record instances
#dbLoadTemplate "db/user.substitutions"
dbLoadRecords "db/eg.db"
dbLoadRecords "db/er.db"

# Enable debugging
EgDebugLevel(5)
ErDebugLevel(5)

## Configure all EVG's
## EgConfigure(X, Y, Z)
## X = Logical card number for this Event Generator card
## Y = VME slot number for this card
```

```
## Z = Starting address for this card's register map
EgConfigure(0, 2, 0x00100000)

## Configure all EVR's
## ErConfigure(V, W, X, Y, Z)
## V = Logical card number for this Event Receiver card
## W = Physical VME slot number for this Event Receiver card
## X = VME A24 address for this card's register map
## Y = Interrupt vector for this card
## Z = VME interrupt request level for this card
ErConfigureVME(0, 4, 0x00110000, 0, 4)
ErConfigureVME(1, 5, 0x00120000, 1, 4)

iocInit
```

Loading the necessary files

Once everything is compiled and you have created a start up script, you will want to copy the necessary files into the correct directory. The RTEMS-running IOC will be set up to look for the boot file and the startup script in a computer acting as its boot server. Our files are currently located in the /srv/tftp directory on that server. The boot file and startup script will be symbolically linked to the file name that the IOC will be looking for. These files will be located in the root of the tftp directory. Your .db files and .dbd files will be put into their own directory. To make life easier you can create a shell script that will handle most of this for you. Here is what a possible script looks like:

```
# Remove current files
rm run.cmd run.ralf st.cmd mrf.boot

# Copy the new files in
cp /home/XXX/epics/timing/mrfApp/iocBoot/iocmrfApp/st.cmd .
cp /home/XXX/epics/timing/mrfApp/bin/RTEMS-mvme3100/mrf.boot .

# Change the group of the files (may not be necessary)
chgrp dialout st.cmd mrf.boot

# Symbolically link the files to the name that the IOC is looking for
ln -s st.cmd run.cmd
ln -s mrf.boot run.ralf

# Again may not be necessary
```

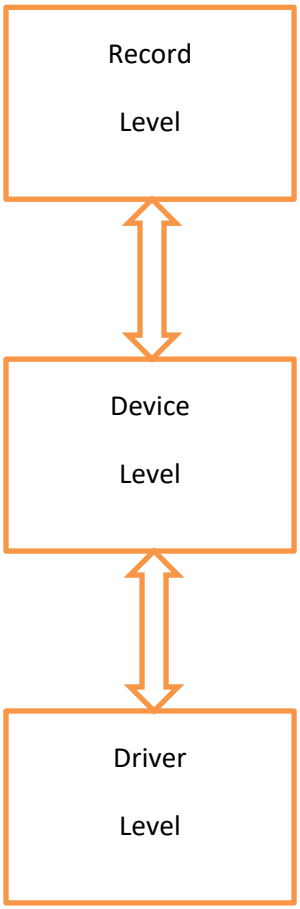
chgrp -h dialout run.cmd run.ralf

You may add commands to copy your .db and .dbd files, but usually this isn't done as often as copying your boot file and start up script. Once the proper files have been moved you may then reboot the IOC and confirm that all files have been loaded properly.

8.2. EPICS

Hierarchy

The device support (application) has three levels of communication:



DB and DBD files that contain the necessary record details of the application you are using

A 'C' file that handles initialization and reading/writing the records

A 'C' file that handles reading/writing from the hardware directly

Each level communicates information to the level above it and requests information from the level below it. The record level, as an example, may request a read from the device level and the device level would then ask the driver level for the appropriate hardware information that the record level is

looking for. This information then would be sent up each level until the record level has received the value.

Timing system record types

The timing system application we are using has four newly created record types. Each record has a number of fields that correspond to various functionalities in the hardware register maps. This is a more monolithic approach than other device supports, which usually have only one record per register functionality.

The new record types are defined here:

Name	Description
eg	Handles all event generator functionalities except for the sequencers
egevent	Handles sequencer functionalities in the event generators
er	Handles all event receiver functionalities except for the mapping RAM
erevent	Handles mapping RAM functionalities

The SharePoint documents detailing the record types should have its own appendix and can be brought up at this point.

Creating a new record field

The device support does not support all of the functionality within the event generators or the event receivers. I believe this is due to a decision by the developers to only support functionality that would be used by other labs. We wanted to test every feature of the hardware and, if necessary, create support if none was currently present.

Each record type is defined in a .dbd file. This file will include all of the fields that have been created for a given record type. Here is a field:

```
field(ENAB, DBF_MENU) {
        prompt("Master enable")
        promptgroup(GUI_DISPLAY)
        menu(menuYesNo)
        pp(TRUE)
        interest(1)
}
```

This field is part of both the eg record type and er record type and controls the master enabler. ENAB is the unique name of the field and DBF_MENU is the type of the field. DBF_MENU means that there will be a menu of options for what the value of this field can be. There are several other field types such as DBF_ULONG which means the field takes a long integer as its value. The menu(menuYesNo) parameter states that the menu options are located in the definition of menuYesNo. Once you find this definition you will see that the value of the ENAB field can either be YES or NO.

As an example, I will explain how to create a field that will allow you to control whether or not data buffer interrupts are enabled in the event receiver. The first thing you must do is add the new field to the record definition in the .dbd file for the event receiver. Here is the new field:

```
field(DINT, DBF_MENU) {
        prompt("Enable the data buffer interrupt?")
        promptdisplay(GUI_DISPLAY)
        menu(menuEventEnable)
        pp(TRUE)
        interest(1)
}
```

You now have a field that will keep track of whether the data buffer interrupt is enabled or disabled. You will now want to initialize this record and get the value from the hardware. Instead of going to the device layer (where the initialization happens) I recommend skipping to the driver layer first. Many times there will already be a function within the device support that will obtain the value you are looking for. Looking at the file **drvMrfEr.c** you will see that there is a function

called **ErDBuffIrq()** that reads the interrupt hardware register to see if data buffer interrupts are enabled. If there is no function in the driver level that meets your needs you will have to create the function yourself.

Open up the file called **devMrfEr.c,** which holds the device layer code. You will see two function of interest: **ErInitRecord()** and **ErProcess()**. The function ErInitRecord() is called on the initialization of an er record. Looking at its code you will see that it relies on calling ErProcess() to read initial values. The ErProcess() function contains numerous function calls to the driver layer. This means that we should create a call to the data buffer interrupt function we have just found. The location of the call doesn't matter, as long as it is while the mutex is locked.

```
ErDBuffIrq(pCard, pRec->dint);
```

Once this is done you can then recompile the application, copy over the boot and dbd file, and reboot the IOC. You will now have a new field to work with.

9. Appendix A

Updating the Sequencer

A method needs to be developed to ensure that the sequencer is updated at an appropriate time and is done as an atomic operation. If the context is switched prior to completion of the write operation then a corrupt table may be used. A method should also be devised to ensure that the running sequence is not modified.

The event sequence is static, as far as I can tell it is loaded at boot time. This needs to be dynamic so that changes can be made on the fly.